

IMPLEMENTATION & USER GUIDE

Meetsy

Turning meeting transcripts into well-scoped, history-grounded, human-confirmed ClickUp tasks — built on the Clicksy data foundation, powered by Azure OpenAI and a per-workspace RAG knowledge base.

Full-proof technical documentation + step-by-step user journey · Phases 0 → 3 complete & live-verified
Generated 2026-06-28 · Branch `feat/meetsy-phase0` · Monorepo: Clicksy (root) + Meetsy
(`apps/meetsy-api` , `apps/meetsy-web`)

Contents

- | | |
|--|---|
| 1 What Meetsy is & the design philosophy | 7 The intelligence layer: prediction, dedup, assignment |
| 2 System architecture & the shared foundation | 8 The learning loop |
| 3 The AI models — which model, which stage, why | 9 End-to-end flow (sequence) |
| 4 The Knowledge Base (RAG) & onboarding | 10 Queues, workers & data stores |
| 5 Documents & the honest improvement metric | 11 User journey guidebook (UI, step by step) |
| 6 The transcript → tasks pipeline (stage by stage) | 12 Honesty & safety principles |
| | 13 Operating it locally (ops appendix) |
-

1 · What Meetsy is & the design philosophy

Clicksy is the existing service that mirrors a company's ClickUp workspace into PostgreSQL (tasks, comments, time entries, status history) for reporting and dashboards. **Meetsy** is a folded-in sibling that uses that mirrored history to do something new: it takes a **meeting transcript** and produces **ClickUp-ready tasks** that are grounded in how the team actually works — then lets a human confirm and push them back into ClickUp.

The two share **one** foundation: one login/session, one organization/workspace model, one PostgreSQL database (Meetsy owns a `meetsy` schema and reads Clicksy's `public` schema read-only), and one ClickUp token. Nothing is forked or duplicated.

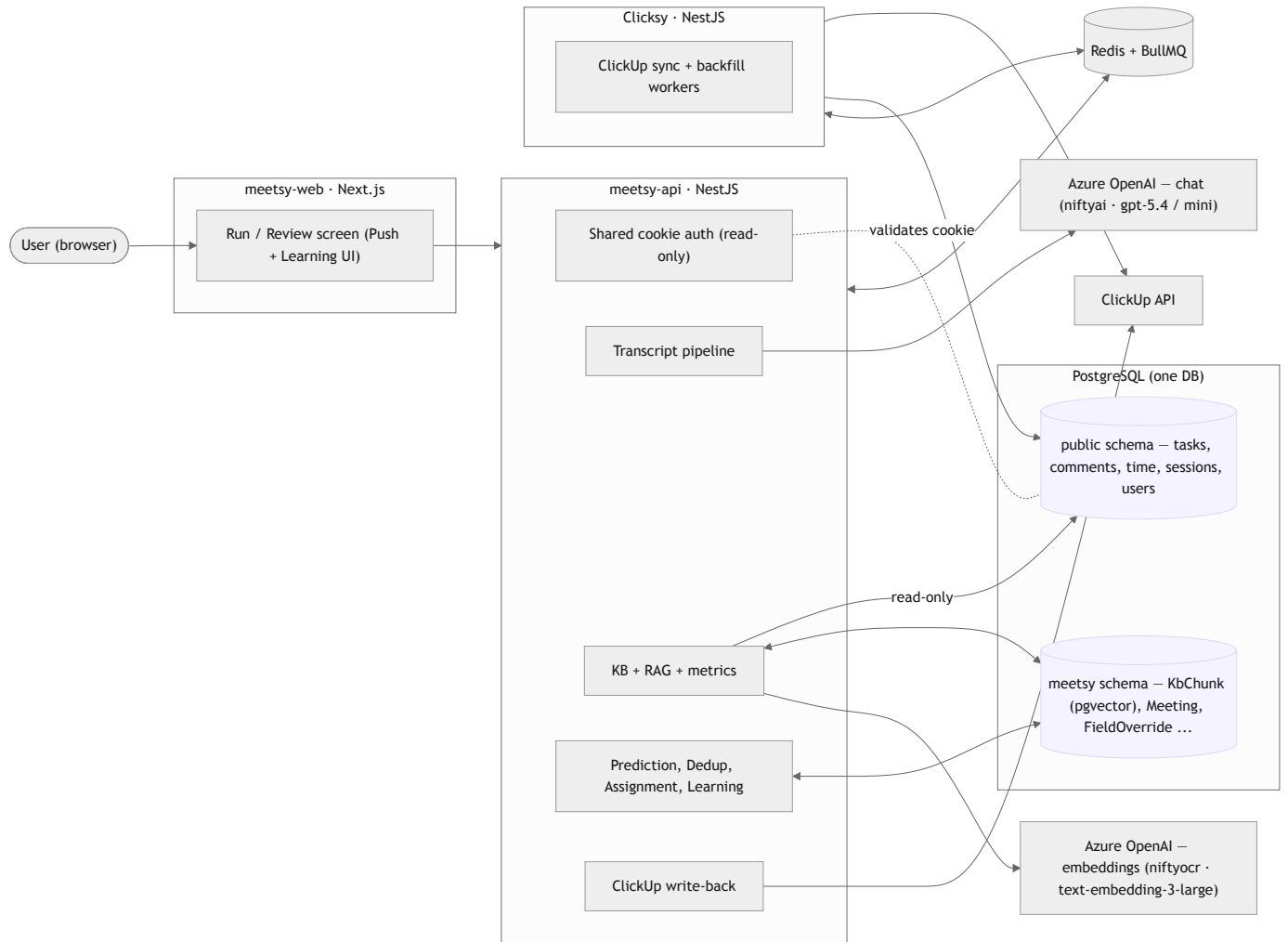
The guiding principle — honesty by construction. Every "smart" output is a **weak, evidence-backed suggestion that abstains when the signal is thin**, never a confident guess and never auto-applied. A human confirms every field before anything is written to ClickUp. Where the system reports "improvement" or "learning", it shows the exact evidence and the sample size, and "not enough data yet" / "no clear answer" is always a valid, first-class result. This principle shaped every component below.

WHAT A USER GETS, IN ONE SENTENCE

Upload a transcript → the pipeline extracts grounded tasks, suggests the client / sprint / due date / owner (or honestly abstains), flags likely duplicates of existing work, and lets you confirm and push them into the right ClickUp list — and the system quietly learns from your corrections over time.

2 · System architecture & the shared foundation

Three deployable apps sit on one shared data plane. The browser app (**meetsy-web**, Next.js) talks to the **meetsy-api** (NestJS); meetsy-api reads Clicksy's mirrored data, calls Azure OpenAI, and pushes to ClickUp. Clicksy owns ingestion from ClickUp.



The shared foundation (Phase 0 & 1)

Concern	How it is shared
Identity / login	One Clicksy cookie session (<code>clickup_sync_sid</code> , SHA-256-hashed token in <code>public.sessions</code>). meetsy-api validates it read-only — there is no separate Meetsy login. One sign-in works across both apps.
Org / workspace / roles	Clicksy's <code>Organization</code> → <code>Workspace</code> model + Owner/Admin/Member roles, enforced app-wide by a global auth guard.
Database	One Postgres. Meetsy owns the <code>meetsy</code> schema (a least-privilege DB role) and reads Clicksy's <code>public.*</code> mirror tables. A DB grant makes writes to <code>public</code> impossible from Meetsy.
ClickUp token	Per-workspace, encrypted at rest (AES-256-GCM) with the same key Clicksy uses. Meetsy only decrypts to read/write tasks.
ClickUp write-back (Phase 1)	Meetsy's own <code>createTask</code> client + per-workspace push config (target list, assignable members), an idempotent audited push (<code>meetsy.TaskPush</code>), and a review screen.

Why read-only on `public` matters: Meetsy can be wrong without ever corrupting Clicksy's source-of-truth mirror. The boundary is enforced by the database itself, not by convention.

3 · The AI models — which model, which stage, why

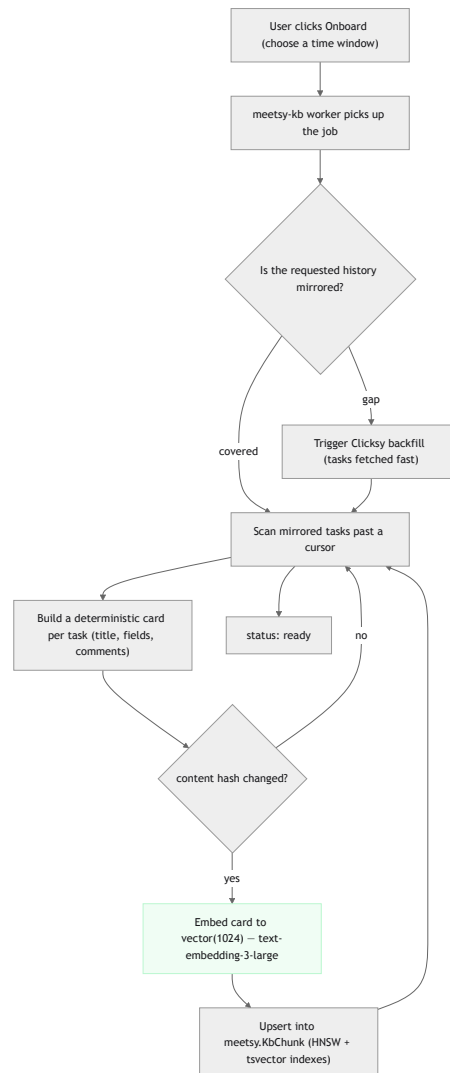
Meetsy uses **Azure OpenAI only**, across **two separate Azure resources**: a chat resource (`niftyai.openai.azure.com`) and a dedicated embeddings resource (`niftyocr.openai.azure.com`). The chat tier is chosen per call: the heavy reasoning stages use **gpt-5.4**; the cheap, bounded, high-volume helpers use **gpt-5.4-mini**. Reasoning effort (low / medium / high) is tuned per stage.

Where	Model	Effort	What it does & why this tier
Stage 0 — Normalize & roster	<code>gpt-5.4</code>	low	Extracts the participant roster from the (deterministically VTT-parsed) transcript. Light task → low effort.
Stage 1+2 — Analyze	<code>gpt-5.4</code>	medium	Comprehends the meeting and extracts grounded candidate tasks (each tied to a verbatim quote). The core reasoning step.
Stage 5 — Critic	<code>gpt-5.4</code>	high	Adversarially verifies grounding, fixes owners, merges duplicates, calibrates confidence. Highest-stakes reasoning → high effort.
Stage 4 — Enrich	<code>gpt-5.4</code>	low	Fills ClickUp-ready fields (acceptance criteria, tags, estimate) and resolves the spoken due-date phrase to an absolute date.
"What we learned" narrative	<code>gpt-5.4-mini</code>	low	One pass that turns exact SQL facts into a short team-profile paragraph (forbidden from inventing numbers). Cheap, summarization-only.
Field-prediction clamp	<code>gpt-5.4-mini</code>	low	Picks a client/sprint/assignee only from values observed in similar past tasks , or abstains. The "echo-breaker" (see §7).
Answerability judge + question extraction	<code>gpt-5.4-mini</code>	low	Derives held-out questions and judges whether retrieved context answers them (doc-improvement metric, §5).
All embeddings	<code>text-embedding-3-large</code>	—	Every task card, document chunk, and query is embedded to a <code>vector(1024)</code> (the 1024-dim option is honored on this model). Powers all RAG retrieval & similarity.

The pattern: reasoning that shapes the actual tasks → **gpt-5.4**. Bounded, high-frequency, judge/clamp/summarize helpers → **gpt-5.4-mini**. Anything that compares meaning → **text-embedding-3-large** + pgvector. The model is selected per call, so cost scales with how much reasoning a step truly needs.

4 · The Knowledge Base (RAG) & onboarding

Before Meetsy can ground anything, a workspace is **onboarded**: its ClickUp history is mirrored (by Clicksy) and embedded into a per-workspace vector index. This is what lets the pipeline answer "how does this team usually describe / scope / own work like this?"



How retrieval works — hybrid search

Every search runs **two branches** and fuses them, so it catches both meaning and exact keywords:

- **Vector branch** — pgvector cosine over the HNSW index (semantic match; paraphrases retrieve the right task even with zero word overlap).
- **Keyword branch** — a generated `tsvector` + Postgres full-text search (exact terms, ids, codes).
- **Fusion** — Reciprocal Rank Fusion (RRF, k=60) merges the two ranked lists into one.

Incremental & cheap: a content-hash per card means a task re-embeds **only** when it actually changes; a per-workspace cursor means re-runs scan only what's new.

The "What we learned" summary card

Right after onboarding, the user sees a profile of the connected team. **Every number is exact aggregate SQL** over the mirrored data (roster & who owns what, components/areas, throughput, blockers, top clients); **one** gpt-5.4-mini pass writes a short narrative on top of those facts (and is

forbidden from inventing figures). Honest by construction — the facts are queryable; only the prose is generated.

Proven on real data: on 1,198 real "Nifty" R&D tasks, the card correctly surfaced the real roster (Rashedul / Shoabur / Ahmad...), real components (R&D Sprints, [Nifty AI] Backlog), and real clients (Nifty AI, Energy Reporting, AIT) — with an accurate generated narrative.

5 • Documents & the honest improvement metric

Users can upload **context documents** (SOPs, scopes, spec PDFs) into a workspace's KB so future analysis is grounded in more than ClickUp history. A document is parsed (text PDFs via `pdf-parse`; OCR is out of scope), chunked (paragraph-aware), and embedded into the same `KbChunk` store as `sourceType=document`.

The hard question — *"did adding this actually make the KB better?"* — is answered with **two signals that are reported separately and never blended**:

① Corpus novelty exact · no LLM

For each chunk, its cosine distance to the nearest existing chunk. "62% of this document is information the KB did not already contain." A near-duplicate honestly scores low.

② Answerability-lift held-out · judged

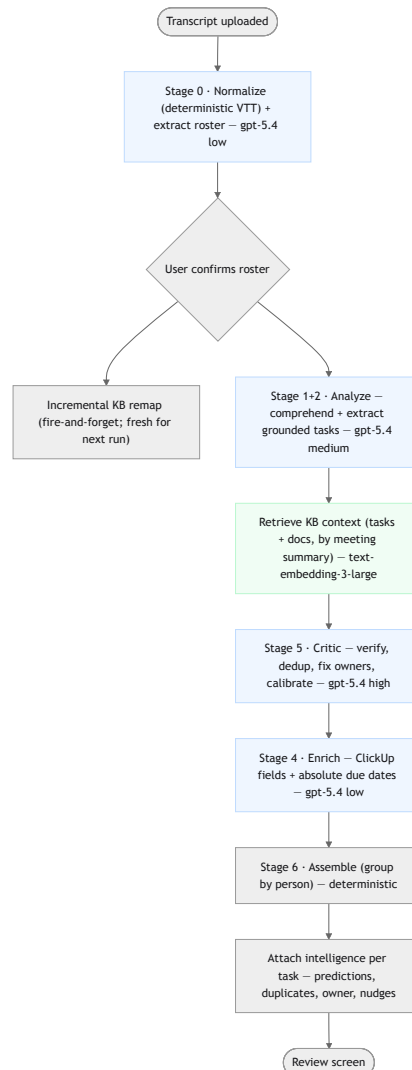
Over questions derived from real meeting transcripts (independent of the doc), how many become answerable *after* the doc is added vs before — judged by a blind, identical gpt-5.4-mini pass on each side.

"No improvement" is a valid result. Novelty is the solid headline; answerability-lift is the proxy. When a workspace has no transcripts yet, answerability falls back to a task-derived baseline that is **clearly labelled provisional**. The system never fuses these into a single "X% better" score.

Live-verified: a real bookkeeping SOP scored meaningfully novel; a vendor-payment policy made the questions "what is our approval threshold?" and "who signs off?" flip from **No** → **Yes** (newly-answerable = 2), while a near-duplicate upload honestly showed ~0 lift.

6 · The transcript → tasks pipeline (stage by stage)

When a transcript is analyzed, it flows through ordered stages. The diagram labels the **AI model at each stage**; the KB and intelligence layers attach **after** the tasks are produced.



Stage	Input → Output	Notes
0 · Normalize	Raw transcript → clean timestamped turns + roster	VTT parsing is deterministic (no model); only the roster extraction calls the LLM.
1+2 · Analyze	Transcript + roster → summary, topics, decisions, candidate tasks	Each task is grounded in a verbatim evidence quote; owners resolved to roster ids only.
Context inject	Summary/topics/titles → top-k history (tasks + docs)	Retrieval is keyed on the concise summary (not the raw transcript). Provenance is kept and shown on the run.
5 · Critic	Candidate tasks (+ context) → corrected, de-duplicated set	Runs before enrich so enrichment applies to the verified set. Drops ungrounded items, merges dupes.
4 · Enrich	Verified tasks (+ context) → ClickUp-ready fields	Resolves "by Wednesday" → an absolute date relative to the meeting date; aligns tags/estimates with house style.
6 · Assemble	Enriched tasks → grouped by assignee + unassigned	Pure. Each task keeps a stable id (<code>t1...tM</code>) that all the per-task signals are keyed by.

7 · The intelligence layer: prediction, dedup, assignment

After the tasks are assembled, each one is enriched with weak, abstain-first signals. All of these reuse one **card-shaped k-NN** over the workspace's task history (so the query is comparable to what's stored), keeping only neighbours above a cosine **floor** — the floor is what makes "abstain" real (a task with no genuinely-similar history simply has too few qualifying neighbours).

Field prediction (client · sprint · due · estimate)



The base-rate echo trap & the fix. A naïve k-NN would just predict the corpus's most common client for everything. Two guards prevent this: (a) the **similarity floor** so out-of-distribution tasks abstain; (b) the **LLM clamp**, which reads the task text and can pull a *minority* value (e.g. "AIT") instead of the majority — while confidence always rides on the real distribution, never the model's say-so. *Verified:* an AIT task predicted **AIT**, not the far more prolific "Energy Reporting"; an out-of-domain task abstained; due dates use the p80 cycle-time (not p50, which here is ≈ "due today").

Duplicate detection

Each task's nearest existing tasks are classified by raw cosine into **flag** ("very likely already exists") and **suggest** ("possibly related"). Bands were **empirically calibrated** against real near-duplicates (a re-created task peaks ~0.73 against its twin, so the literal-spec 0.90 would never fire). It **never auto-merges** — purely advisory.

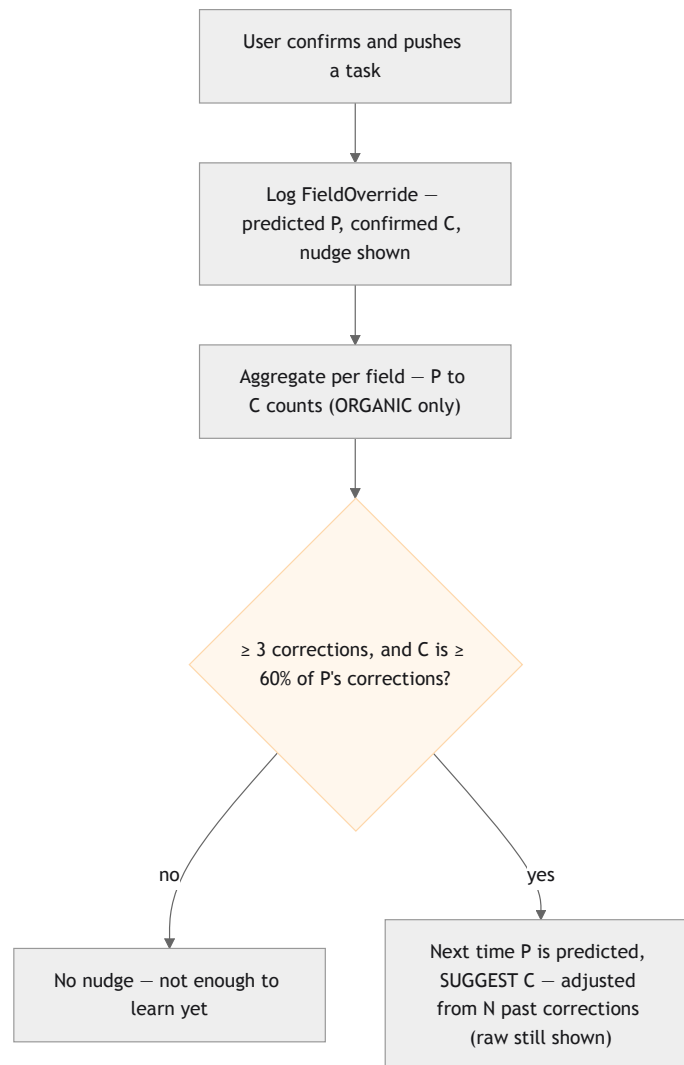
Smart assignment (Phase 3.1)

For each task, candidates from the workspace's **assignable pool** are ranked by **ownership precedent** — who closed similar work, weighted by similarity (closed > open), **conditioned on the predicted client/area** so a minority-area task gets its real owner, not the busiest contributor. **Workload** (open tasks + tracked hours) only breaks near-ties; it never assigns someone with no precedent. It **abstains** when there's no clear owner, and names an out-of-pool owner rather than silently giving up. *Verified:* energy task → Rashedul; AIT task → Ahmad (not the prolific Rashedul); off-topic task → abstain.

8 · The learning loop

Every push records what the model **predicted** versus what the human **confirmed** (a **FieldOverride** row), plus what nudge the loop showed. When users **consistently** correct the model the same way,

future predictions are nudged toward the corrected value — gated, labelled, and honestly measured. It is a **deterministic correction memory, not a trained model**.



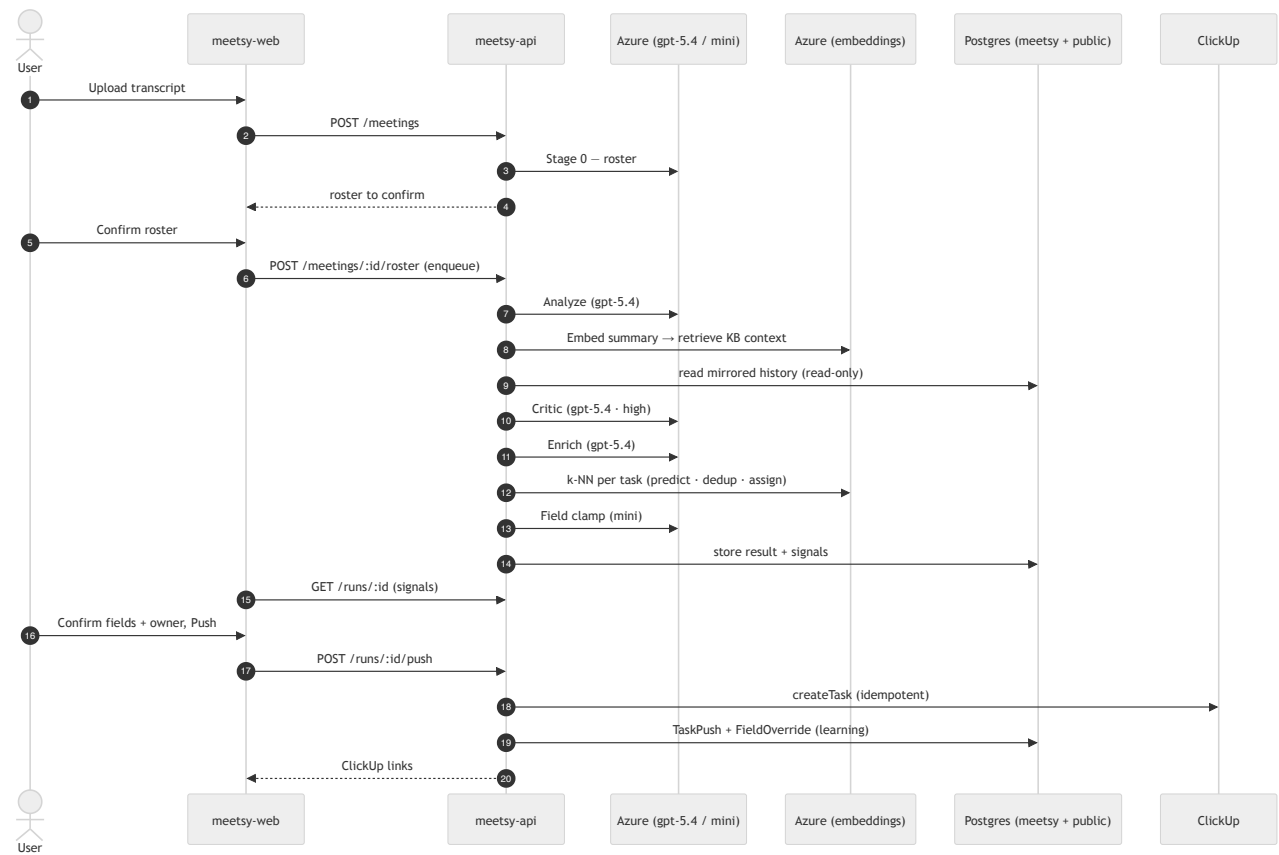
Two anti-traps make the loop honest.

- **Organic-only gate.** A correction made *after* a nudge was shown does **not** teach the gate — otherwise an accepted nudge would reinforce the very rule that produced it (a self-fulfilling loop).
- **Two separate metrics.** The *raw override rate* (how often you change the model) measures KB/model quality — it can't measure the loop, because an accepted nudge still differs from the raw prediction. So the loop's *real* effectiveness is the **nudge-acceptance rate**, reported separately, with sample sizes shown and never blended.

Verified, both directions: the gate fires at 3 consistent corrections (and the nudge appears on the next run), and does **not** fire at 2 corrections or a 50/50 split. Unresolved values are counted so a resolution bug can't masquerade as "sparse data".

9 · End-to-end flow (sequence)

The full life of one transcript, from upload to a pushed ClickUp task and the resulting learning signal.



10 · Queues, workers & data stores

BACKGROUND WORKERS (REDIS + BULLMQ)

Queue	App	Job
analysis	meetsy-api	The transcript pipeline (analyze → critic → enrich → assemble → intelligence).
meetsy-kb	meetsy-api	KB onboarding/refresh: coverage check → backfill trigger → incremental embed. Crash-safe (short lock + stalled-recovery + supersede-on-enqueue).
meetsy-kb-docs	meetsy-api	Document ingest: parse → chunk → embed → novelty + answerability + doc↔task links.
clickup-backfills, clickup-tasks, clickup-time-entries, clickup-comments	Clicksy	Mirror ClickUp into public.* (the data Meetsy reads).

KEY TABLES IN THE MEETSY SCHEMA

Table	Role
KbChunk	One embedded card per task/document chunk: <code>vector(1024)</code> + a generated <code>tsvector</code> + filter metadata. The RAG index.
KbSyncState · KbSummary	Onboarding cursor/status · the cached "what we learned" card.
KbDocument · KbDocTaskLink	Uploaded docs (extracted text + the honest metric) · doc↔task relatedness.
Meeting · AnalysisRun	The uploaded transcript + confirmed roster · the pipeline result (incl. all per-task signals).
WorkspacePushConfig · TaskPush	Target list, assignable members, client/sprint/points options · the idempotent push audit ledger.
Field0override	predicted vs confirmed (+ the shown nudge) per pushed task — the learning loop's input.

11 · User journey guidebook (UI, step by step)

This is the end-to-end experience for a user turning a meeting into ClickUp tasks. Screens are in the Meetsy web app; you are already signed in via the shared Clicksy login.

- 1 **Sign in (once, shared).** Use your normal Clicksy login. The session works across both apps — there is no separate Meetsy account. If your session expires, any action redirects you back to the Clicksy sign-in.
- 2 **Onboard the workspace's knowledge base.** Choose a history window and start onboarding. Meetsy mirrors and embeds the workspace's ClickUp tasks. A live status shows progress; when it reads **ready**, the KB is searchable. (One-time per workspace; it stays fresh automatically and refreshes before each analysis.)
- 3 **Read the "What we've learned" card.** Immediately after onboarding you see a profile of the team: who owns which areas, the main components, throughput, top clients, where work stalls — exact numbers, plus a short narrative. This is your confidence check that the right history is connected.
- 4 **(Optional) Upload context documents.** Add SOPs / scopes / spec PDFs. Each upload shows how **novel** it is vs what the KB already knew, which existing tasks it relates to, and (once you have transcripts) its answerability-lift. A near-duplicate honestly shows little novelty.
- 5 **Configure push settings.** On *Push settings*, set the target ClickUp list and the assignable members. On the review screen you can press **"Refresh ClickUp fields"** to pull the live client dropdown options and sprint lists so they appear as proper pickers.
- 6 **Upload a transcript & confirm the roster.** Paste/upload the meeting transcript. Stage 0 extracts the participant roster; you confirm or correct it, then the analysis is enqueued.
- 7 **Watch the pipeline run.** A live stepper shows Normalize → Comprehend → Extract → Critic → Enrich → Assemble. When it completes, the review screen loads.

8 Review the result — read the signals. Each task card shows the AI's weak, abstain-first suggestions (all reference, nothing applied yet):

- **Grouded in N items** a banner showing the history the analysis was grouded in.
- **Suggested: Client · Sprint · Due · Estimate** with a "· N similar" confidence cue, or "—" when it abstains.
- **Possibly related / Likely already exists** duplicate flags against existing ClickUp tasks.
- **Owner: Suggested <name> · N closed similar** the ownership-based recommendation (or "no clear owner").
- **Adjusted client: X → Y · from N past corrections** a learning-loop nudge, when one has been earned.

Analysis

New analysis

Run cmqxdg55t0002g79bk1o4v2in · status Completed

OVERVIEW

Sarah identified a bug in the Energy Reporting web portal audit dashboard where Audit Objectives shows the wrong default, said the Energy Audit Web Portal still needs to be built, and assigned Tom to fix the default issue and verify the portal PDF generation against the AEA gold standard by Friday. Tom is not in the provided roster, so no assigneeld can be set for those tasks.

► Grounded in 8 items of this client's history (8 tasks)

TASKS BY PERSON

No assigned tasks.

UNASSIGNED

Tasks the pipeline couldn't confidently assign to anyone.

Fix the Audit Objectives default in the Energy Reporting web portal audit dashboard

High

SUGGESTED Client: Energy Reporting · 15 similar Sprint: Sprint 11 (16/3/26 - 22/3/26) · 2 similar Due: 2026-07-01 Estimate: —
Adjusted client: Energy Reporting → AIT · from 3 past corrections
OWNER Suggested: Rashedul Hasan · 8 closed similar

Correct the wrong default shown for Audit Objectives in the Energy Reporting web portal audit dashboard.

Due: 2026-07-03 Estimate: 4h

energy-reporting web-portal audit-dashboard bugfix audit-objectives

ACCEPTANCE CRITERIA

- Audit Objectives displays the correct default value on initial load of the Energy Reporting web portal audit dashboard.
- The corrected default remains consistent after page refresh and when reopening the dashboard.
- No regression is introduced to related audit dashboard fields or selection behavior.
- The fix is deployed or committed and validated in the target test environment.

SUBTASKS

- Reproduce the incorrect default behavior in the audit dashboard.
- Update the default-value logic/configuration for Audit Objectives.
- Test initial load, refresh, and reopen scenarios to confirm the fix.

↳ Evidence (2)

Helpful?  

Verify Energy Audit Web Portal PDF generation against the AEA gold-standard

High

SUGGESTED Client: Energy Reporting · 15 similar Sprint: Energy Reporting · 3 similar Due: 2026-06-29 Estimate: —
Adjusted client: Energy Reporting → AIT · from 3 past corrections
OWNER Suggested: Rashedul Hasan · 8 closed similar

Verify that the Energy Audit Web Portal PDF generation matches the AEA gold-standard.

Due: 2026-07-03 Estimate: 4h

energy-reporting web-portal pdf verification aea

ACCEPTANCE CRITERIA

- Generate a PDF from the Energy Audit Web Portal using representative audit data.
- Compare the generated PDF against the AEA gold standard and document any differences.
- Confirm all required sections, values, formatting, and ordering match the AEA gold standard, or log any mismatches for follow-up.
- Share or attach verification results showing whether the PDF generation passes validation.

SUBTASKS

- Generate a sample PDF from the portal.
- Perform a side-by-side comparison with the AEA gold standard.
- Record discrepancies or confirm compliance.

DEPENDENCIES

- Build the Energy Audit Web Portal

↳ Evidence (1)

Helpful?  

Build the Energy Audit Web Portal

Normal

Possibly related: 86evkrgvw Possibly related: 86evkejtz
SUGGESTED Client: Energy Reporting · 15 similar Sprint: R&D Sprint 10 (17/11/25 - 23/11/25) · 4 similar Due: 2026-07-02 Estimate: —
Adjusted client: Energy Reporting → AIT · from 3 past corrections
OWNER Suggested: Rashedul Hasan · 8 closed similar

Build the Energy Audit Web Portal; no owner or deadline was specified in the meeting.

Due: : null Estimate: : null

energy-reporting web-portal engineering build

ACCEPTANCE CRITERIA

- The Energy Audit Web Portal is implemented and accessible in the intended environment.
- Core portal workflows needed for audit use are functional without blocking errors.
- The portal is connected to the required data inputs/services needed for normal operation.
- Basic stakeholder or QA validation confirms the portal is ready for follow-on testing such as PDF verification.

SUBTASKS

- Define the required portal scope from the discussed audit use case.
- Implement the portal screens and core workflow.
- Connect required backend/data integrations.
- Run basic functional validation and address blocking issues.

• Evidence (1)

Helpful?  

How feedback works:  = correct ·  with a comment = we revise it ·  with no comment = it's removed · no vote = accepted by default.

Submit feedback

Push to ClickUp

Target list: R&D Default

Refresh ClickUp fields

Push to ClickUp (3)

☒ Fix the Audit Objectives default in the Energy Reporting web portal audit dashboard

Assignee	Priority	Due date	Sprint / list	Client	Points
Rashedul Hasan	High	03/07/2026	Default (R&D Default)	Energy Reporting	—

☒ Verify Energy Audit Web Portal PDF generation against the AEA gold-standard

Assignee	Priority	Due date	Sprint / list	Client	Points
Rashedul Hasan	High	03/07/2026	Default (R&D Default)	Energy Reporting	—

☒ Build the Energy Audit Web Portal

Assignee	Priority	Due date	Sprint / list	Client	Points
Rashedul Hasan	Normal	dd/mm/yyyy	Default (R&D Default)	Energy Reporting	—

► What we've learned (3 past pushes)

Recover missed tasks

Notice a missing task? Tell me, e.g. "You missed the task about the database migration."

No messages yet.

e.g. You missed the task about the database migration.

Send

The real review screen (captured in an automated browser test): KB-context banner, per-task suggestion / duplicate / owner / learning chips, and the push controls with pre-filled assignee, sprint, client and points.

9 Confirm fields & owner, then push. In the Push section each task row is editable: assignee (pre-filled from the ownership recommendation), priority, due date, **sprint/list**, **client** (pre-filled from the prediction), and **points**. Adjust anything, tick the tasks to include, and press "**Push to ClickUp**". Tasks land in the right list with the chosen fields; pushes are idempotent (re-pushing never duplicates).

10 See it improve over time. Open "**What we've learned**" to see the corrections the system has aggregated and the two honest metrics. As you keep confirming/correcting, consistent patterns (≥ 3 times, $\geq 60\%$ agreement) start appearing as labelled nudges on future runs — and the panel honestly says "not enough to learn yet" until then.

What stays in your hands: nothing is assigned, adjusted, or pushed without your click. Every suggestion shows its evidence, and the system prefers to say "not sure" over guessing.

12 · Honesty & safety principles

Principle	How it shows up
Abstain-first	A similarity floor + minimum support means thin/out-of-distribution inputs honestly say "not enough history" instead of guessing.
Recommendation, never automation	Predictions, owners, and nudges are defaults a human confirms. No auto-assign, no auto-push, no auto-merge of duplicates.
Evidence on everything	Suggestions show the count of similar tasks; assignment cites closed-similar work; nudges cite the number of past corrections; retrieval shows provenance.
Never blend metrics	Doc novelty vs answerability-lift, and raw-override-rate vs nudge-acceptance, are always reported separately with sample sizes.
Support-gated learning	The loop nudges only after ≥ 3 consistent corrections with $\geq 60\%$ agreement; "not enough to learn yet" is a valid state.
Read-only on the source of truth	Meetsy can never corrupt Clicksy's mirror — the database grant forbids writes to <code>public</code> .
Idempotent & audited writes	Every ClickUp push is recorded in <code>TaskPush</code> ; a re-push of an already-pushed task is a no-op, not a duplicate.

Production safety. The ClickUp token is least-privilege and per-workspace; verification work is performed only against a throwaway test team, never a production workspace; webhook signatures are verified; and a wrong Meetsy output is, by construction, never able to damage the customer's real ClickUp source data.

13 · Operating it locally (ops appendix)

For engineers running the stack. The full, current runbook lives in `docs/meetsy/RESUME.md` and `docs/meetsy/BUILD-JOURNAL.md`.

STACK

- **PostgreSQL** on the `pgvector/pgvector:pg18` image (the `vector` extension is required) + **Redis**.
- **meetsy-api** (NestJS, Node ≥ 22.12 , Prisma 7) on port 4000; **meetsy-web** (Next.js) on port 3000; **Clicksy** on port 3000-range for ingestion.
- **Azure OpenAI**: a chat resource (gpt-5.4 / gpt-5.4-mini) and a separate embeddings resource (text-embedding-3-large).

BRING-UP ORDER (THE OPERATOR FLOW)

1. Run `prisma/grants.sql` as superuser → `CREATE EXTENSION vector`, the least-priv 'meetsy' role + schema, `search_path`, `SELECT` grants on the public mirror tables.
2. Apply the meetsy migrations (they contain `NO CREATE SCHEMA / CREATE EXTENSION`).
3. Set `env: MEETSY_DATABASE_URL, REDIS_*, AZURE_OPENAI_* (chat), AZURE_EMBED_*`

(embeddings), APP_ENCRYPTION_KEY, optional CLICKSY_ADMIN_URL + ADMIN_API_KEY.
4. Start meetsy-api (dist/main) and meetsy-web (next start). Sign in via Clicksy.

CORE ENVIRONMENT VARIABLES

Variable	Purpose
MEETSY_DATABASE_URL	The least-privilege meetsy role on the shared Postgres (? schema=meetsy).
AZURE_OPENAI_*	Chat resource + deployment (gpt-5.4). Reasoning effort is set per call in code.
AZURE_EMBED_*	Embeddings resource (text-embedding-3-large, dimensions 1024).
APP_ENCRYPTION_KEY	Decrypts the per-workspace ClickUp token (same key Clicksy encrypts with).
CLICKSY_ADMIN_URL + ADMIN_API_KEY	Lets KB onboarding trigger a Clicksy backfill for missing history (degrades gracefully if unset).

ROADMAP STATUS

Phases 0 → 3 are complete and live-verified on real data: shared auth/foundation → ClickUp write-back → RAG KB + summary + document metric → pipeline grounding (context, prediction, dedup, HITL push) → smart assignment + the learning loop, with the review UI surfacing it all (and browser-verified). Open follow-ups: injecting KB context into the very first analyze stage, tuning the novelty/dedup thresholds on more data, and Clicksy's per-org data isolation.